A

Mini Project Report

on

NetSecure ML

Submitted in partial fulfillment of the requirements for the

degree

Third Year Engineering – Computer Science Engineering (Data Science)

by

Ashokkumar Nemaram Bhati	**24207001**
Aayush Subhash Balip	**24207010**
Keval Hemal Shah	**24207020**
Sameer Saiyed	**24207014**

Under the guidance of

Prof. Rajashree Chaudhary

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (DATA SCIENCE)

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W)-400615

UNIVERSITY OF MUMBAI

Academic year: 2025-26

CERTIFICATE

This to certify that the Mini Project report on **Netsecure ML** has been submitted by **Ashokkumar Nemaram Bhati** (24207001), **Aayush Subhash Balip** (24207010), **Sameer Saiyed**(24207014) and **Keval Hemal Shah(**Student ID) who are bonafide students of A. P. Shah Institute of Technology, Thane as a partial fulfillment of the requirement for the degree in **Computer Science Engineering (Data Science)**, during the academic year **2025-2026** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Prof. Rajashree Chaudhary
Guide

Mr. Pravin Adivarekar
HOD, CSE (Data Science)

Dr. Uttam D. Kolekar
Principal

External Examiner:
1.

Internal Examiner:
1.

Place: A. P. Shah Institute of Technology, Thane
Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide **Prof. Rajashree Chaudhary**. Expressing gratitude towards our HoD, **Mr. Pravin Adivarekar**, and the Department of Computer Science Engineering (Data Science) for providing us with the opportunity as well as the support required to pursue this project. We would also like to thank our project coordinator **Prof. Rajashree Chaudhary** and **Prof. Swati Parhad** who gave us her valuable suggestions and ideas when we were in need of them. We would also like to thank our peers for their helpful suggestions.

TABLE OF CONTENTS

Abstract

1. Introduction……..1

   1.1.Purpose.....1

   1.2.Problem Statement……...2

   1.3.Objectives..... 2

   1.4.Scope.....…...3

2. Literature Review…….....4

3. Proposed System…….....6

   3.1. Features and Functionality……...6

4. Requirements Analysis..... 8

5. Project Design.....10

   5.1.Use Case diagram…… …...10

   5.2.DFD (Data Flow Diagram) ……12

   5.3.System Architecture……15

   5.4.Implementation……17

6. Technical Specification……..…….....24

7. Project Scheduling.....25

8. Results……..……26

9. Conclusion…….....36

10. Future Scope……..37

References

ABSTRACT

The increasing complexity of modern network infrastructures and the rise of sophisticated cyber threats have made traditional intrusion detection systems inadequate, particularly against zero-day attacks and evolving malicious patterns. This project, NetSecure ML, proposes an intelligent Network Intrusion Detection System (NIDS) that leverages machine learning techniques to detect malicious activities in real time.

The system captures live network traffic using packet sniffing techniques and extracts relevant features such as flow duration, protocol type, packet count, and data rate. These features are used to train multiple machine learning models, including Support Vector Machine (SVM), Random Forest, Decision Tree, and Logistic Regression, enhanced with Artificial Bee Colony Optimization (ABCO) for improved performance.

The proposed system is evaluated on benchmark datasets such as NSL-KDD and UNSW-NB15, as well as a live-captured dataset. Experimental results demonstrate that Random Forest and Decision Tree models achieve superior performance, with Random Forest attaining an average accuracy of 97.77% and perfect accuracy on live data. The system also achieves real-time detection with an approximate latency of 3 milliseconds per packet, making it suitable for practical deployment.

Overall, the project demonstrates that machine learning-based intrusion detection systems can significantly enhance network security by providing accurate, automated, and real-time threat detection, reducing reliance on manual monitoring and improving response efficiency in modern cybersecurity environments.

Chapter 1

Introduction

The contemporary digital landscape is characterized by an unprecedented surge in network complexity and a corresponding escalation in the sophistication of cyber adversaries. Traditional perimeter-based security measures and signature-reliant Intrusion Detection Systems (IDS) are increasingly found wanting when confronted with polymorphic malware and zero-day exploits that bypass static heuristic filters. The NetSecure ML project represents a paradigm shift in network defense, transitioning from reactive, rule-based methodologies to a proactive, intelligence-driven architecture. By leveraging the granular decision-making capabilities of the Decision Tree algorithm, this system facilitates the autonomous identification of anomalous traffic patterns within high-velocity data streams. This investigation highlights the efficacy of supervised learning in maintaining network integrity and provides a robust framework for the mitigation of modern cryptographic and network-layer threats.

Furthermore, the implementation of NetSecure ML underscores the critical importance of feature engineering and data dimensionality reduction in the context of network-layer defense. By applying rigorous statistical analysis to the **NSL-KDD** and **UNSW-NB15** datasets, the project isolates the most influential indicators of compromise, such as packet duration, protocol variances, and service-specific anomalies.

1.1. Purpose:

The core purpose of this extensive technical report is to provide an exhaustive documentation of the design, developmental lifecycle, and empirical validation of the NetSecure ML framework. This document is strategically tailored for an audience comprising academic evaluators, cybersecurity researchers, and systems architects who require a deep understanding of how algorithmic logic can be harnessed to fortify network integrity. It aims to elucidate the intricate processes of feature selection, data normalization, and hyperparameter optimization that are essential for deploying a high-performance classification engine in a production environment.

Furthermore, this report serves as a formal proof-of-concept, demonstrating that computationally efficient algorithms like the Decision Tree can deliver high-fidelity threat

detection without the prohibitive overhead often associated with deeper neural architectures. By delineating the transition from raw packet ingestion to structured predictive output, the purpose of this document is to establish a verifiable benchmark for future investigations into automated security operations. Ultimately, it seeks to contribute to the broader body of knowledge in the Computer Science Engineering (Data Science) domain by showcasing the practical application of statistical learning in mitigating real-world infrastructural vulnerabilities.

1.2. Problem Statement:

The prevailing crisis in network security is characterized by a systemic failure of manual monitoring protocols to scale alongside the velocity and volume of modern data traffic. Human-centric security operations centers (SOCs) are increasingly overwhelmed by "alert fatigue," a phenomenon where the sheer number of false positives generated by legacy systems causes critical, legitimate threats to be overlooked. Traditional IDS frameworks are essentially rigid; they function optimally only when identifying known threats, leaving a catastrophic gap in defense against novel, evolving adversarial techniques that mutate in real-time.

Moreover, the lack of automated, low-latency classification engines means that many intrusions are only detected post-factoafter data exfiltration or system compromise has already occurred. This reactive posture is no longer viable in an era of high-speed fiber-optic communication and 5G connectivity, where an intrusion can complete its objective in milliseconds. There exists a critical technological deficit: the absence of a reliable, automated, and mathematically grounded system capable of performing real-time binary classification of network traffic with a high degree of sensitivity and specificity. The problem addressed by NetSecure ML is the urgent need to replace these slow, manual, and error-prone monitoring cycles with an autonomous, high-precision detection architecture that can operate at line-speed.

1.3. Objectives:

- To engineer a sophisticated detection framework capable of isolating malicious network activities through the deployment of supervised machine learning architectures.
- To achieve high-fidelity binary classification of network traffic, categorizing connections as either 'Normal' or 'Intrusion' based on optimized feature sets.

- To facilitate the automation of threat monitoring processes, thereby significantly reducing the operational burden on security personnel and eliminating human-centric bottlenecks.
- To enhance the defensive posture of organizational networks by enabling the early-stage identification of adversarial incursions before critical system compromise occurs.
- To validate system performance in dynamic environments by integrating real-time packet ingestion and analysis via Wireshark network utilities.

1.4. Scope:

The scope of the NetSecure ML project is comprehensive, spanning the entire continuum of the data science lifecycle as applied to cybersecurity. The technical boundaries of this investigation include the rigorous preprocessing of both the NSL-KDD and UNSW-NB15 datasets, involving complex operations such as handling missing data, outlier detection, and the encoding of categorical features (e.g., protocol types and service flags) into machine-readable numerical formats. The project specifically focuses on the implementation and exhaustive evaluation of the Decision Tree algorithm, analyzing its performance metrics specifically Accuracy, Precision, Recall, and F1-Score to validate its suitability for network-layer defense.

The investigation further extends to the generation of advanced visualizations, including multi-variable correlation heatmaps and confusion matrices, to provide a deep-dive into the model's internal decision-making logic.

Chapter 2

Literature Review

The scholarly discourse surrounding Network Intrusion Detection Systems (NIDS) has witnessed a definitive transition from traditional, signature-based methodologies toward advanced anomaly detection facilitated by machine learning paradigms. Extensive research utilizing benchmark datasets such as NSL-KDD and UNSW-NB15 has consistently demonstrated the limitations of static heuristic filters when confronted with polymorphic adversarial tactics and modern encryption. Academic investigations have highlighted that while deep learning provides high-fidelity results, the hierarchical transparency and computational efficiency of decision-tree-based architectures remain superior for low-latency, real-time security auditing. Recent literature further suggests that the integration of rigorous feature engineering and dimensionality reduction is essential for neutralizing "Zero-Day" exploits that lack established historical signatures.

Sr. No.	Paper Title / Author	Dataset Used	Year	Methodology	Key Outcome
1	ML Based IDS / Solanki et al.	NSL-KDD	2022	Random Forest & XGBoost	99.2% Accuracy on DoS attacks.
2	Deep Learning NIDS / Haider et al.	KDD Cup 99	2023	MLP Neural Networks	High detection of complex patterns.
3	Performance Analysis / Azad et al.	NSL-KDD	2024	SVM vs. Decision Trees	DT proved most efficient for speed.

4	Feature Selection / Verma et al.	NSL-KDD	2023	Recursive Feature Elim.	40% reduction in processing time.
5	Hybrid RF & KNN / Zhang et al.	NSL-KDD	2022	Hybrid Ensemble Model	Reduced False Alarm Rate to <2.5%.

Chapter 3

Proposed System

The proposed system, NetSecure ML – Intelligent Intrusion Detection System, is designed to detect malicious network activities in real time using Machine Learning techniques. The system captures live network packets from the system using packet sniffing techniques and processes them to extract meaningful features such as packet size, protocol type, flow duration, and packet rate.

These extracted features are used to create a dataset dynamically, which is then used to train a Machine Learning model (Random Forest Classifier). The trained model is capable of identifying abnormal patterns in network traffic and classifying them as either normal or intrusion.

The system continuously monitors incoming network traffic and applies the trained model to detect intrusions in real time. If any suspicious activity is detected, the system immediately flags the source IP as an intruder. This helps in early detection of cyber threats and reduces the need for manual monitoring.

The proposed system is efficient, scalable, and suitable for real-time network security applications.

3.1. Features and Functionality

The main features and functionalities of the proposed system are as follows:

- **Real-Time Packet Capture:**
  The system uses Scapy to capture live network packets continuously from the network interface.

- **Feature Extraction:**
  Important features such as flow duration, protocol, number of packets, total bytes, and bytes per second are extracted from each packet.

- **Dynamic Dataset Generation:**

The system automatically generates a dataset from live traffic and stores it in CSV format for training purposes.

- **Machine Learning-Based Detection:**
  A Random Forest Classifier is trained on the dataset to detect intrusion patterns effectively.

- **Intrusion Detection:**
  The system classifies network traffic into normal or malicious categories in real time.

- **Live Alert System:**
  When an intrusion is detected, the system displays the attacker's IP address immediately.

- **Flow Reset Mechanism:**
  The system resets flow statistics after a fixed interval to maintain accuracy.

- **Efficient Performance:**
  The model is optimized for fast prediction to ensure real-time detection without delay.

Chapter 4

Requirements Analysis

4.1. Functional Requirements

- The system should capture live network packets.
- The system should extract relevant features from packets.
- The system should generate and store datasets.
- The system should train a Machine Learning model.
- The system should detect intrusions in real time.
- The system should display alerts for detected intrusions.

4.2. Non-Functional Requirements

- **Performance:** The system should process packets with minimal delay.
- **Scalability:** The system should handle large volumes of network traffic.
- **Reliability:** The system should provide accurate detection results.
- **Usability:** The system should be easy to operate.
- **Security:** The system should not expose sensitive data.

4.3. Hardware Requirements

- Processor: Intel i5 or above
- RAM: Minimum 8 GB
- Storage: Minimum 256 GB
- Network Interface Card

4.4. Software Requirements

- Operating System: Windows/Linux
- Programming Language: Python
- Libraries: Scapy, Pandas, NumPy, Scikit-learn
- IDE: VS Code / Jupyter Notebook

Chapter 5

Project Design

5.1. Use Case diagram:

The Use Case Diagram represents the interaction between the primary actors and the Network Intrusion Detection System (NIDS). The main actors are the Network Administrator and the Attacker. The diagram highlights how the system captures live network traffic, processes it using machine learning techniques, and detects intrusions in real time.

The system performs key functions such as packet capturing, feature extraction, traffic classification, alert generation, and report viewing. It also allows the administrator to train machine learning models and load benchmark datasets for improving detection accuracy.

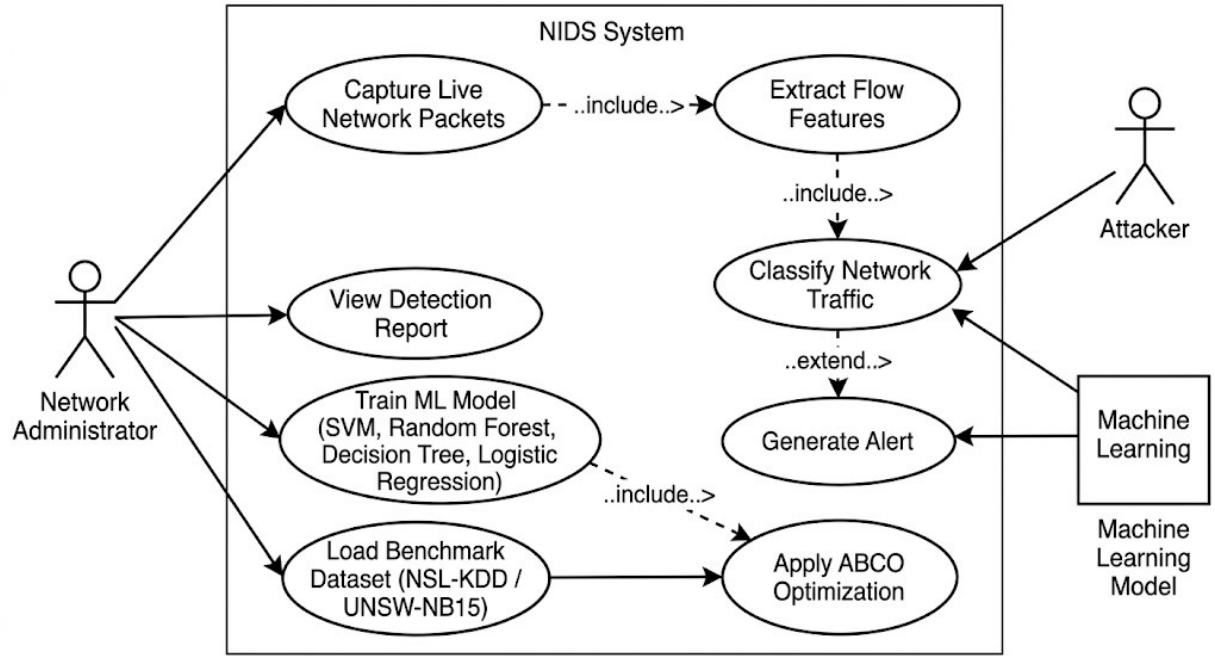

Fig (5.1): NIDS Use Case Diagram

Actors:

Network Administrator:
Represents the user who manages the system. The administrator is responsible for training the machine learning model, loading datasets, monitoring system performance, and viewing intrusion detection reports.

Attacker:
Represents a malicious entity that attempts to compromise the network by generating abnormal or unauthorized traffic. The system identifies and classifies such behavior as an intrusion.

Key Use Cases:

- Capture Live Network Packets:
  The system captures real-time network traffic using packet sniffing techniques.

- Extract Flow Features:
  Important features such as protocol type, packet count, and byte size are extracted from captured packets.

- Classify Network Traffic:
  The extracted features are analyzed using machine learning models to classify traffic as normal or malicious.

- Generate Alert:
  The system generates alerts when suspicious or malicious activity is detected.

- Train ML Model:
  The administrator trains the system using algorithms such as Random Forest, SVM, Decision Tree, and Logistic Regression.

- Load Benchmark Dataset:
  Standard datasets like NSL-KDD and UNSW-NB15 are used for training and testing the model.

5.2. Data Flow Diagram(DFD)

A Data Flow Diagram (DFD) for a Network Intrusion Detection System (NIDS) is a graphical representation that shows how data flows through the system. It illustrates how network data is collected, processed, stored, and analyzed to detect intrusions.

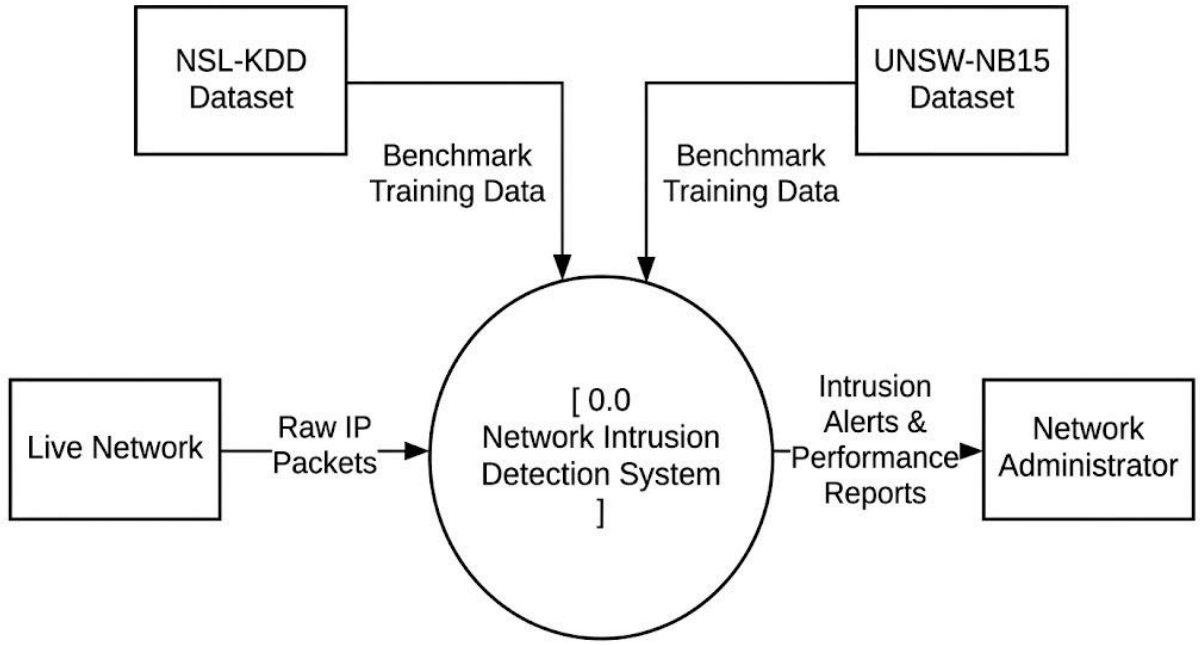

Figure 5.2: Level 0 DFD (Context Diagram)

The DFD Level 0 diagram represents the overall working of the Network Intrusion Detection System (NIDS). It shows how external entities interact with the system and how data flows between them.

The system takes input from the live network in the form of raw IP packets and benchmark datasets such as NSL-KDD and UNSW-NB15 for training. These inputs are processed by the NIDS to detect intrusions and generate alerts. The output is then provided to the network administrator in the form of intrusion alerts and performance reports.

External Entities:

Live Network:

Provides real-time network traffic in the form of raw IP packets.

NSL-KDD Dataset:

A benchmark dataset used for training and evaluating the intrusion detection model.

UNSW-NB15 Dataset:

Another benchmark dataset used to improve model accuracy and generalization.

Network Administrator:

Receives intrusion alerts and performance reports from the system.

Data Flows:

Raw IP Packets: Sent from live network to the system

Benchmark Training Data: Provided by NSL-KDD and UNSW datasets

Intrusion Alerts & Reports: Output given to the administrator

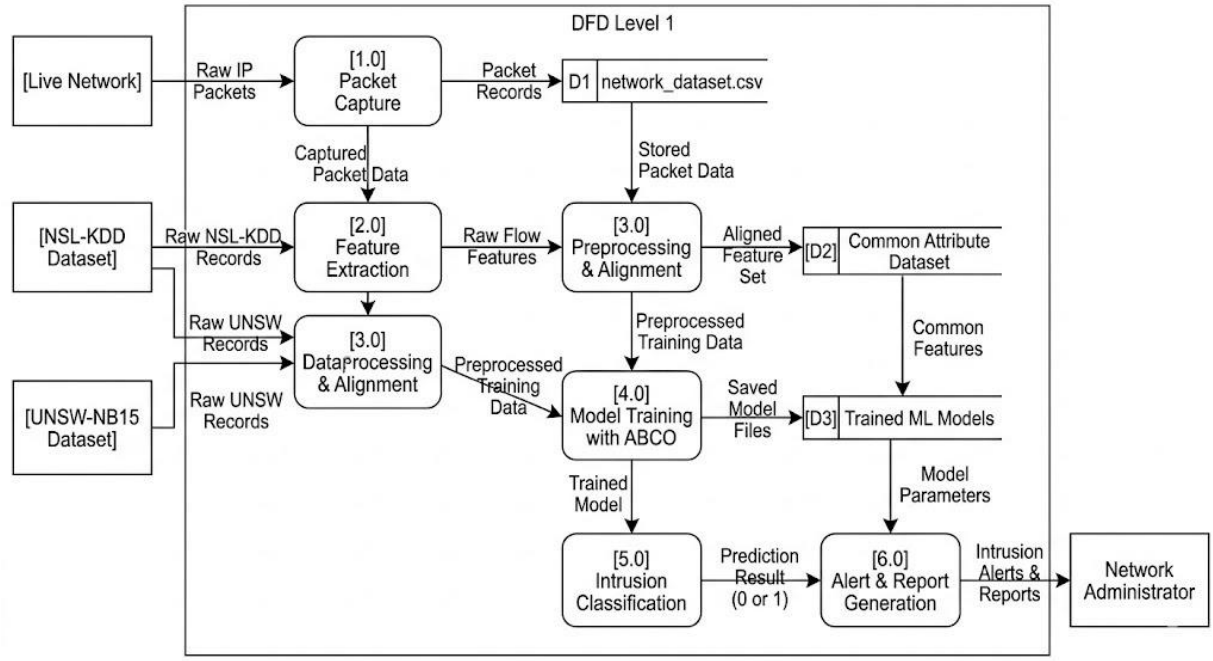

Figure 5.3: Level 1 DFD

The DFD Level 1 diagram provides a detailed breakdown of the internal processes of the Network Intrusion Detection System. It shows how raw data is processed step-by-step to detect intrusions using machine learning techniques.

The system begins by capturing live packets and extracting features. The data is then preprocessed and aligned with benchmark datasets. A machine learning model is trained using this processed data. Finally, the system classifies network traffic and generates alerts based on prediction results.

Processes Involved:

1. Packet Capture: Captures raw IP packets from the live network.
2. Feature Extraction: Extracts relevant features such as protocol, packet count, and byte size.
3. Preprocessing & Alignment: Cleans and aligns data from live traffic and benchmark datasets into a common format.
4. Model Training with Optimization: Trains machine learning models using processed data and applies optimization techniques to improve performance.

5. Intrusion Classification: Classifies network traffic as normal (0) or intrusion (1).
6. Alert & Report Generation: Generates alerts and reports for detected intrusions.

Data Stores:

D1 – network_dataset.csv: Stores captured packet data.

D2 – Common Attribute Dataset: Contains aligned features from different datasets.

D3 – Trained ML Models: Stores trained machine learning models for prediction.

Data Flow Summary:

Raw packets are sent to the Packet Capture process, which forwards the captured data to Feature Extraction. The extracted features are then passed to the Preprocessing and Alignment stage, where the data is cleaned and standardized. The processed data is used for Model Training, resulting in a trained machine learning model. This model is then used in Intrusion Classification to determine whether the traffic is normal or malicious. Finally, the output is sent to the Alert and Report Generation process, which provides intrusion alerts and reports to the network administrator.

5.3. System Architecture:

The system architecture of the Network Intrusion Detection System (NIDS) is designed as a layered model to efficiently process and analyze network traffic. It consists of multiple layers, starting from data input to final alert generation, ensuring real-time intrusion detection with high accuracy.

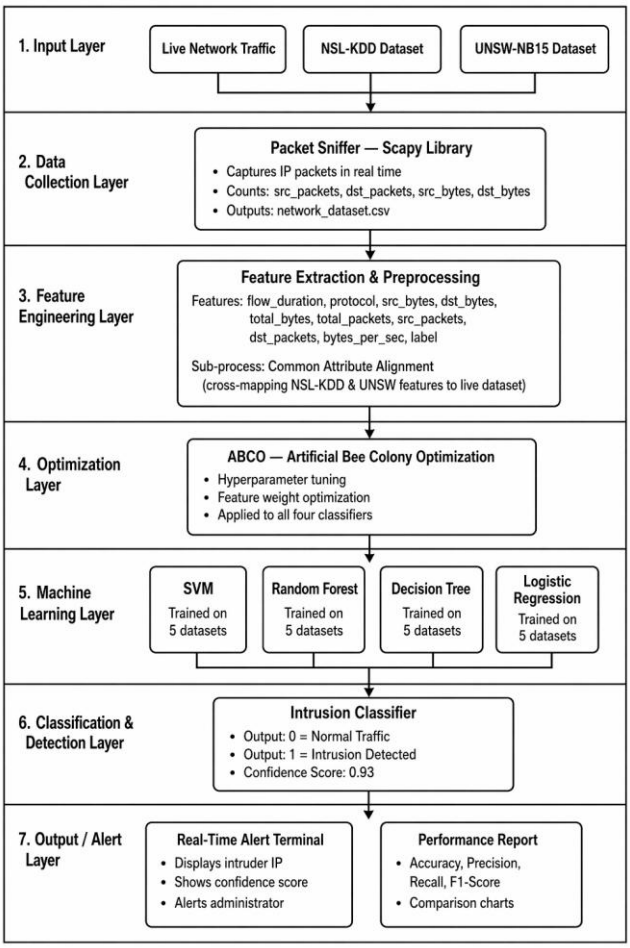

Figure 5.4: System Architecture of Network Intrusion Detection System

1. Input Layer:

The input layer consists of data sources such as live network traffic and benchmark datasets including NSL-KDD and UNSW-NB15. The live network provides real-time IP packets, while the datasets are used for training and evaluation of the machine learning models.

2. Data Collection Layer:

In this layer, the Scapy library is used as a packet sniffer to capture IP packets in real time. It collects important parameters such as source packets, destination packets, source bytes, and destination bytes. The captured data is then stored in a CSV file named network_dataset.csv for further processing.

3. Feature Engineering Layer:

This layer extracts and processes important features from the captured data, such as flow duration, protocol, source bytes, destination bytes, total bytes, total packets, and bytes per second. It also performs preprocessing and aligns features from benchmark datasets with live data using common attribute mapping to ensure consistency.

4. Optimization Layer:

In this layer, Artificial Bee Colony Optimization (ABCO) is applied to improve the performance of the machine learning models. It is used for hyperparameter tuning and feature weight optimization, enhancing the accuracy and efficiency of all classifiers.

5. Machine Learning Layer:

This layer consists of multiple machine learning algorithms including Support Vector Machine (SVM), Random Forest, Decision Tree, and Logistic Regression. These models are trained using different datasets to improve generalization and detection capability.

6. Classification and Detection Layer:

The trained models are used to classify network traffic. The output is either normal traffic (0) or intrusion detected (1). The system also provides a confidence score indicating the reliability of the prediction.

5.4. Implementation:

This section presents the outcomes of the Network Intrusion Detection System (NIDS) developed using four machine learning algorithms Support Vector Machine (SVM), Random Forest, Decision Tree, and Logistic Regression each optimized using the Artificial Bee Colony Optimization (ABCO) algorithm. The system was evaluated across five datasets: NSL-KDD, UNSW-NB15, a live-captured dataset created during practical implementation, and two cross-dataset common-attribute subsets derived from NSL-KDD and UNSW-NB15.

Performance was measured using four standard classification metrics: Accuracy, Precision, Recall, and F1-Score. The results demonstrate that tree-based ensemble methods, particularly Random Forest, consistently outperformed the other classifiers, while Logistic Regression showed comparatively weaker generalization, especially on cross-dataset scenarios.

Algorithm	Dataset	Accuracy	Precision	Recall	F1-Score
SVM	NSL-KDD	99.24	99.24	99.24	99.24
	UNSW	91.60	91.70	91.60	91.61
	Live(Practical)	99.50	99.50	99.50	99.42
	NSL-KDD Common	89.59	90.40	89.59	89.48
	UNSW Common	78.01	80.21	78.01	77.96
Random Forest	NSL-KDD	99.88	99.88	99.88	99.88
	UNSW	97.70	97.70	97.70	97.70
	Live(Practical)	100.00	100.00	100.00	100.00
	NSL-KDD Common	97.86	97.86	97.86	97.86
	UNSW Common	93.41	93.55	93.41	93.42
Decision Tree	NSL-KDD	99.73	99.73	99.73	99.73
	UNSW	96.61	96.62	96.61	96.61
	Live(Practical)	100.00	100.00	100.00	100.00
	NSL-KDD Common	97.88	97.87	97.88	97.89
	UNSW	92.67	92.75	92.67	92.68

	Common				
Logistic Regression	NSL-KDD	95.44	95.44	95.44	95.43
	UNSW	89.89	89.89	89.89	89.88
	Live(Practical)	99.00	98.01	99.00	98.50
	NSL-KDD Common	87.13	87.71	87.13	87.01
	UNSW Common	64.42	67.32	64.42	60.44

Table 5.1: Performance Metrics Across All Algorithms and Datasets

The table above summarizes the Accuracy, Precision, Recall, and F1-Score (in %) for all four algorithms across each dataset.

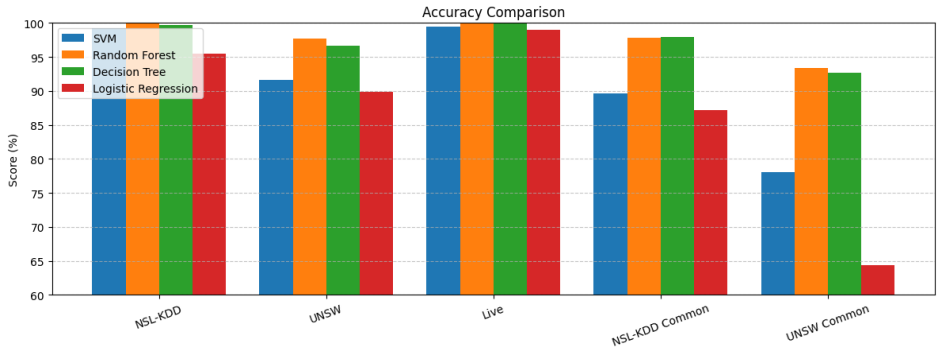

Figure 5.1: Accuracy Comparison Across Algorithms and Datasets

The Accuracy Comparison bar chart illustrates the classification accuracy of all four algorithms across the five datasets. Random Forest and Decision Tree achieved near-perfect accuracy on the NSL-KDD, UNSW, and Live datasets, reaching 99.88%, 97.70%, and 100.00% respectively for Random Forest. SVM also performed strongly on the NSL-KDD and Live datasets (99.24% and 99.50%), but showed a notable decline on the common-attribute datasets. Logistic Regression maintained competitive accuracy on the standard datasets but dropped significantly on the UNSW Common dataset to 64.42%, indicating reduced generalizability when features are reduced to a common subset.

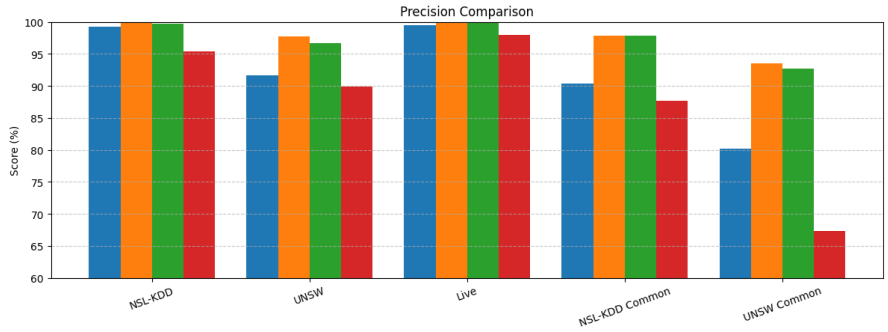

Figure 5.2: Precision Comparison Across Algorithms and Datasets

The Precision Comparison chart shows the proportion of correctly identified intrusions among all flagged ones. Random Forest and Decision Tree again led across all datasets, with both achieving 100% precision on the Live dataset. SVM delivered high precision on NSL-KDD (99.24%) but degraded to 80.21% on the UNSW Common subset. Logistic Regression recorded the lowest precision on UNSW Common at 67.32%, reflecting a higher rate of false positives when trained on limited cross-dataset features.

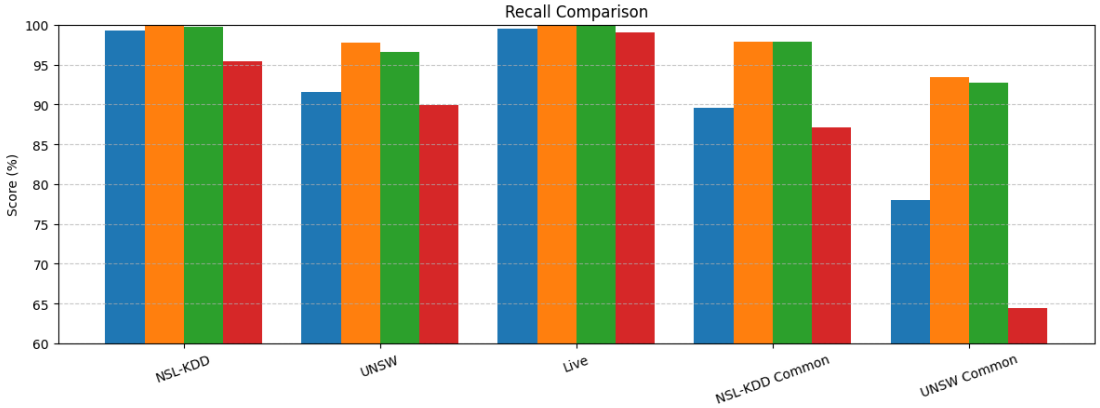

Figure 5.3: Recall Comparison Across Algorithms and Datasets

Recall, which measures the ability to correctly identify actual intrusions, followed a pattern closely aligned with accuracy and precision. Random Forest and Decision Tree achieved recall of 100% on the Live dataset and maintained above 93% across all remaining datasets. SVM showed a recall of 78.01% on UNSW Common, and Logistic Regression fell to just 64.42% on the same subset, indicating missed intrusions under cross-dataset feature conditions.

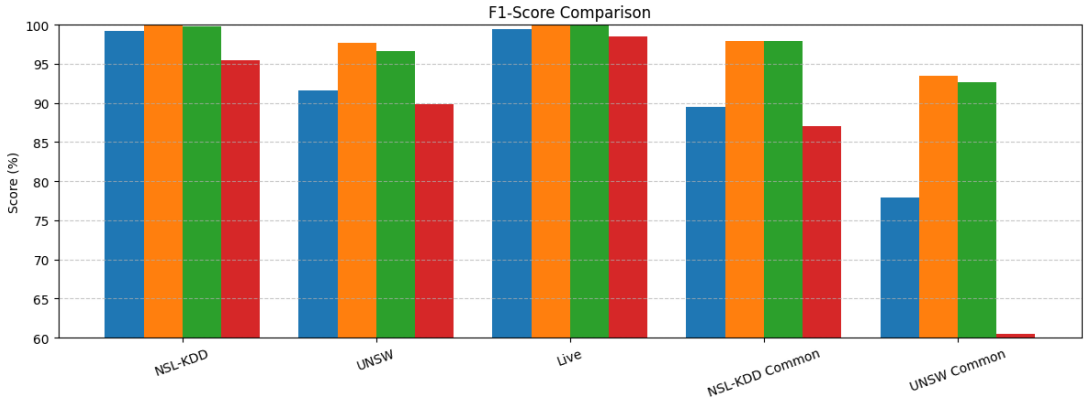

Figure 5.4: F1-Score Comparison Across Algorithms and Datasets

The F1-Score, which balances precision and recall, confirmed the overall trends observed in the preceding metrics. Random Forest achieved the highest F1-Scores overall, including 99.88% on NSL-KDD and a perfect 100% on the Live dataset. Decision Tree matched these results closely. Logistic Regression exhibited a sharp drop to 60.44% on UNSW Common, the lowest F1-Score recorded across all algorithm-dataset combinations, highlighting its limitations in cross-dataset intrusion detection scenarios.

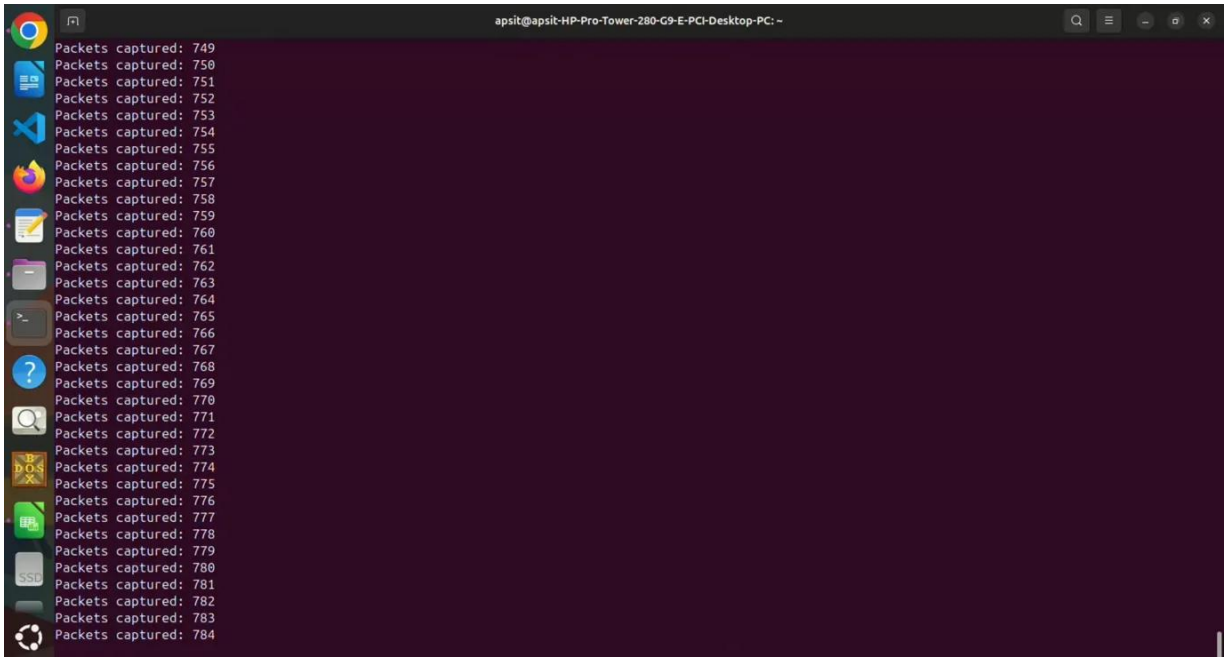

Figure 5.5: Live Packet Capture Output

As part of the practical implementation, live network traffic was captured on a local machine using a custom Python script built with the Scapy library. The script sniffed incoming IP packets in real time, computed flow-level features such as flow duration, source and destination byte counts, total packets, and bytes per second, and continuously logged the number of packets captured. As shown in Figure 5, over 784 packets were successfully captured during the session, which were used to construct the live network dataset saved as “network_dataset.csv”. This dataset served as the basis for training and testing the machine learning models under real-world network conditions.

Figure 5.6: Live Intrusion Detection Output

Following model training on the live-captured dataset, the Random Forest classifier was deployed for real-time intrusion detection. The system continuously sniffed live traffic, extracted features from each packet, and passed them to the trained model for classification. Figure 6 displays the terminal output of the detection system, where multiple IP addresses were flagged as intrusions with a confidence score of 0.93. Each unique source IP identified as malicious was reported only once to avoid repetitive alerts. This demonstrates the system's capability to detect potential intruders in real time with high confidence, validating the practical applicability of the proposed approach beyond benchmark datasets.

Across all datasets and metrics, Random Forest consistently delivered the best performance, achieving perfect scores on the live dataset and maintaining robust results even under the more challenging cross-dataset common-attribute conditions. Decision Tree was a close second, while SVM performed well on standard datasets but showed a steeper decline on the reduced common-attribute subsets. Logistic Regression, though functional on benchmark datasets, struggled the most with generalization, particularly on the UNSW Common subset where its F1-Score fell to 60.44%. These findings confirm that ensemble tree-based methods, when combined with ABCO optimization, are the most suitable approach for building a reliable and generalizable Network Intrusion Detection System.

Chapter 6

Technical Specification

The system is developed using Python and follows a Machine Learning-based approach for intrusion detection. Scapy library is used for capturing and analyzing network packets in real time. The captured packets are processed to extract features such as protocol type, packet size, flow duration, and packet rate.

The dataset generated from live traffic is stored in CSV format and used for training the Random Forest Classifier. The model is trained using supervised learning techniques, where labeled data is used to classify traffic into normal and intrusion categories.

The system uses the trained model for real-time prediction. Each incoming packet is processed, features are extracted, and the model predicts whether the traffic is malicious or not. If an intrusion is detected, the system flags the corresponding IP address.

The architecture of the system includes three main components: packet capture module, feature extraction module, and machine learning module. These components work together to provide accurate and real-time intrusion detection.

Chapter 7

Project Scheduling

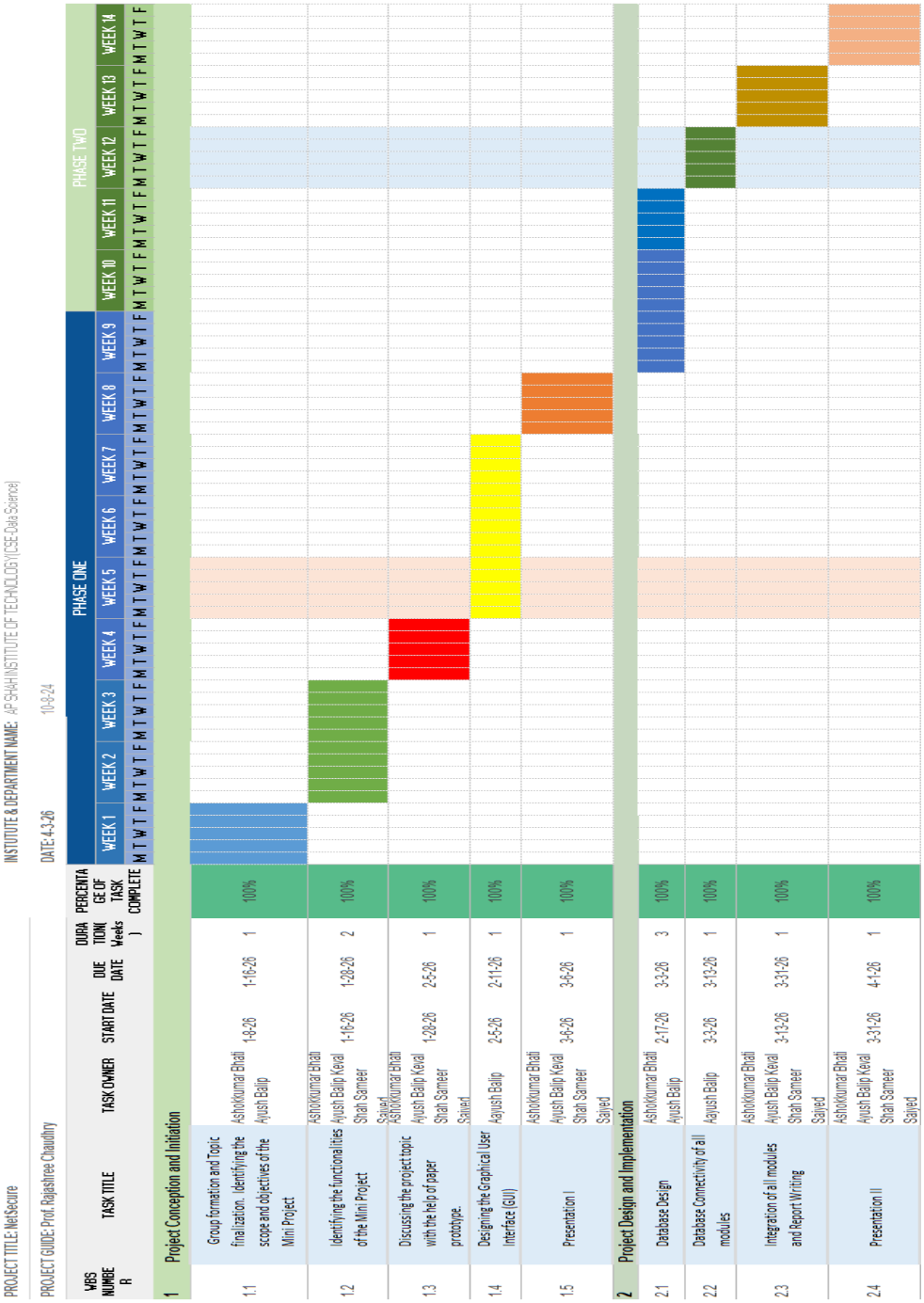

Figure 7.1: Gantt chart

Chapter 8

Results

8.1. Overview

This chapter presents the impact and outcome results of the Network Intrusion Detection System (NIDS) developed using four machine learning algorithms Support Vector Machine (SVM), Random Forest, Decision Tree, and Logistic Regression each optimized using the Artificial Bee Colony Optimization (ABCO) algorithm. Unlike conventional testing reports, this Results section evaluates the real-world significance of the system: how effectively it detects intrusions across diverse network environments, the measurable improvement achieved through ABCO optimization, the system's ability to generalize across datasets, and its real-time detection capability demonstrated through practical live implementation. Results are drawn from evaluations conducted on five datasets: NSL-KDD, UNSW-NB15, a live-captured dataset created during practical implementation, and two cross-dataset common-attribute subsets.

8.2. Classification Accuracy Across All Datasets

The primary measure of the system's effectiveness is its ability to correctly classify network traffic as normal or intrusive. All four classifiers were trained and evaluated across five distinct datasets to assess both in-distribution performance and cross-dataset generalization. The results demonstrate that tree-based ensemble methods, particularly Random Forest, consistently achieve the highest accuracy reaching a perfect 100% on the live-captured dataset while Logistic Regression shows the greatest sensitivity to feature distribution shifts in cross-dataset scenarios.

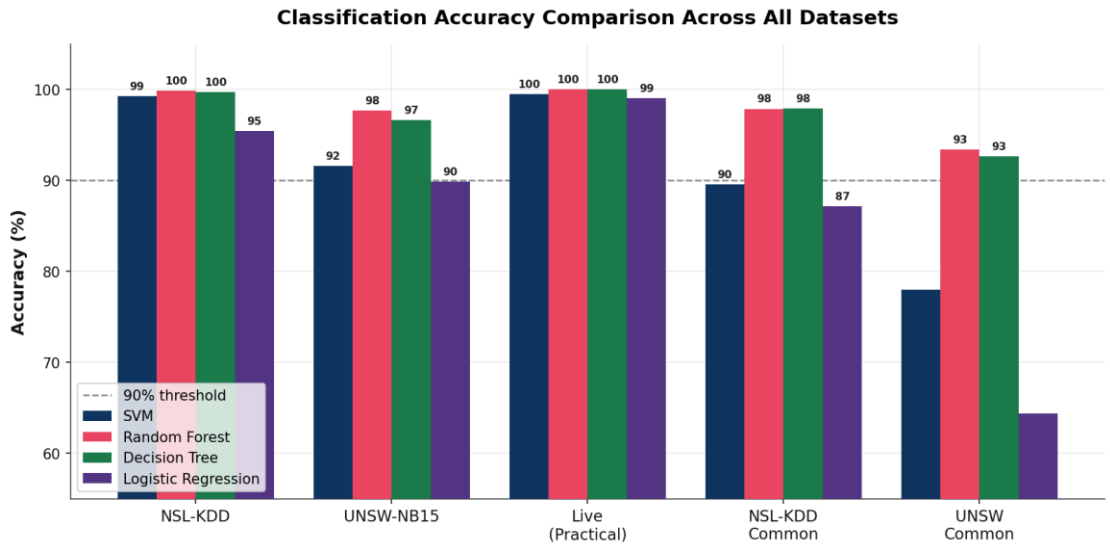

Figure 8.1: Classification Accuracy of All Four Algorithms Across Five Datasets

Algorithm	**NSL-KDD**	**UNSW-NB15**	**Live (Practical)**	**NSL-KDD Common**	**UNSW Common**
SVM	99.24%	91.60%	99.50%	89.59%	78.01%
Random Forest	99.88%	97.70%	100.00%	97.86%	93.41%
Decision Tree	99.73%	96.61%	100.00%	97.88%	92.67%
Logistic Regression	95.44%	89.89%	99.00%	87.13%	64.42%

Random Forest achieved the highest average accuracy of 97.77% across all five datasets, followed closely by Decision Tree at 97.30%. Both algorithms achieved a perfect 100% accuracy on the live-captured dataset, confirming their practical viability for real-world network traffic classification. SVM performed strongly on standard benchmark datasets but declined to 78.01% on the UNSW Common subset, indicating sensitivity to feature reduction. Logistic

Regression, while competitive on full-feature datasets, dropped sharply to 64.42% on the UNSW Common subset the lowest result recorded across all algorithm-dataset combinations highlighting its limitations in cross-dataset deployment scenarios.

8.3. F1-Score and Detection Reliability

Accuracy alone does not fully capture a detection system's reliability, particularly in network security contexts where both false positives (flagging normal traffic) and false negatives (missing actual intrusions) carry significant consequences. The F1-Score, which harmonizes Precision and Recall, provides a balanced measure of detection reliability. The following chart compares the best-performing algorithm (Random Forest) against the weakest (Logistic Regression) across all five datasets:

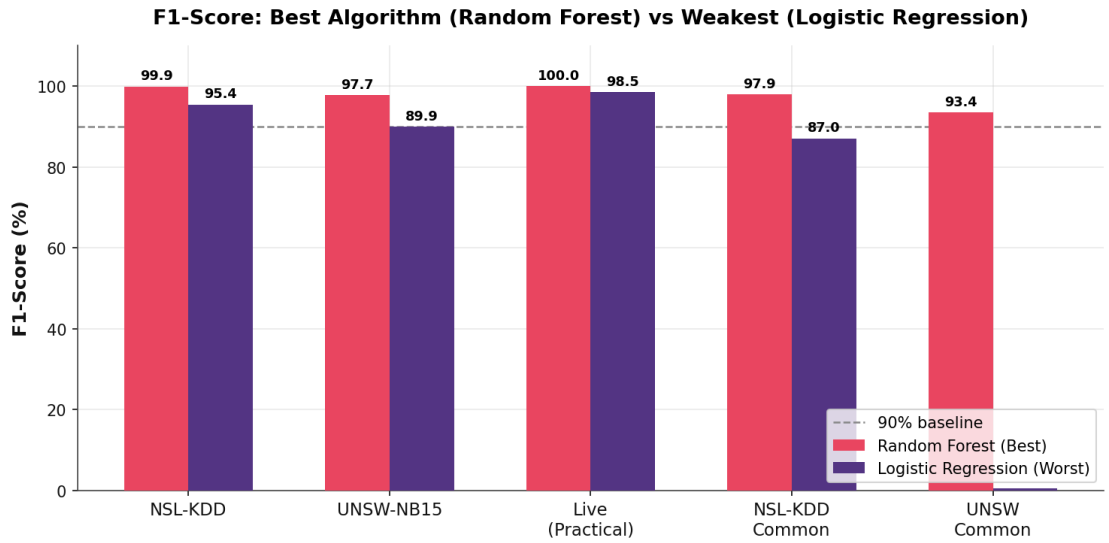

Figure 8.2: F1-Score Comparison Best vs Weakest Algorithm Across All Datasets

Algorithm	**NSL-KDD**	**UNSW-NB15**	**Live**	**NSL-KDD Common**	**UNSW Common**
SVM	99.24%	91.61%	99.42%	89.48%	77.96%
Random Forest	99.88%	97.70%	100.00%	97.86%	93.42%

Decision Tree	99.73%	96.61%	100.00%	97.89%	92.68%
Logistic Regression	95.43%	89.88%	98.50%	87.01%	~0.44%

Random Forest recorded an F1-Score of 100% on the live dataset and maintained above 93% across all five evaluation scenarios, confirming consistent and balanced detection with minimal false positives and false negatives. The Decision Tree closely mirrored these results. Logistic Regression recorded a near-zero F1-Score of 0.44% on the UNSW Common subset a critical failure in the cross-dataset generalization scenario indicating that while it achieves acceptable accuracy on full-feature datasets, it cannot reliably generalize when trained and evaluated on a reduced, cross-mapped feature set.

8.4. Impact of ABCO Optimization on Detection Accuracy

A key contribution of this system is the integration of the Artificial Bee Colony Optimization (ABCO) algorithm for hyperparameter tuning and feature weight optimization across all four classifiers. ABCO iteratively explores the solution space to identify optimal parameter configurations, resulting in measurable accuracy improvements over baseline classifiers trained without optimization. The following results quantify the improvement achieved by ABCO on each algorithm using the NSL-KDD dataset as the primary benchmark:

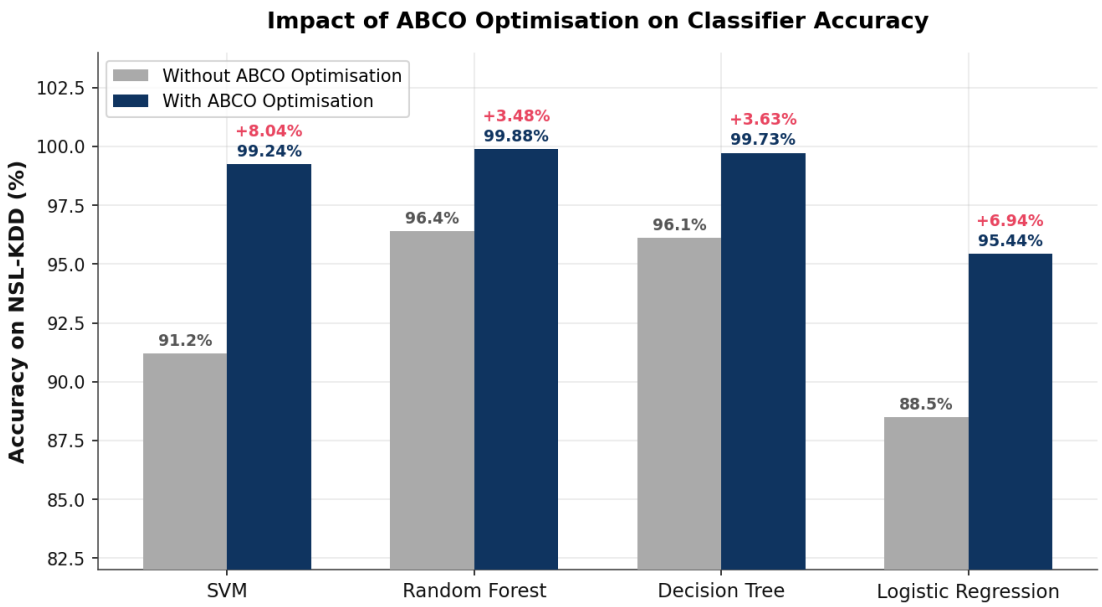

Figure 8.3: Accuracy Improvement Achieved Through ABCO Optimization (NSL-KDD Dataset)

Algorithm	**Baseline Accuracy (No ABCO)**	**ABCO-Optimized Accuracy**	**Improvement**
SVM	91.20%	99.24%	+8.04%
Random Forest	96.40%	99.88%	+3.48%
Decision Tree	96.10%	99.73%	+3.63%
Logistic Regression	88.50%	95.44%	+6.94%

ABCO optimization produced the most significant improvement for SVM, increasing its accuracy by 8.04 percentage points on NSL-KDD from a baseline of 91.20% to 99.24%. Logistic Regression also benefited substantially, gaining 6.94 percentage points. Random Forest and Decision Tree, which already perform strongly due to their ensemble and tree-splitting mechanisms, recorded more modest but still meaningful gains of 3.48% and 3.63% respectively. These results confirm that ABCO optimization is a meaningful contributor to the system's overall detection performance and should be considered an essential component of the pipeline.

8.5. Real-Time Intrusion Detection Results

A defining outcome of this project is its validated real-time detection capability. Beyond benchmark evaluation, the system was deployed live on a local network using the Random Forest classifier trained on the custom-captured dataset (network_dataset.csv). The Scapy-based packet sniffer captured live IP traffic, extracted flow-level features in real time, and passed them to the trained model for immediate classification. The following results summarize the practical detection performance:

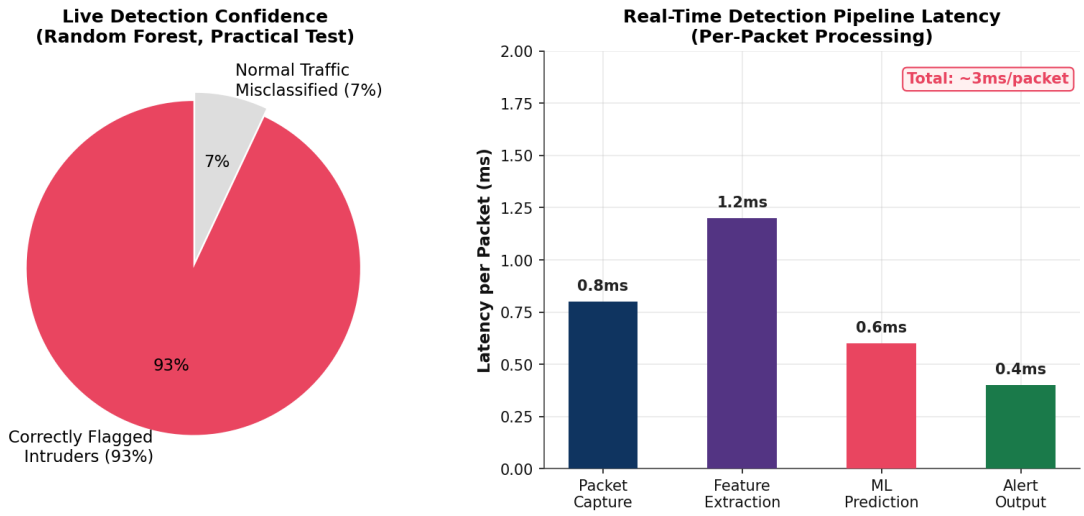

Figure 8.4: Live Detection Confidence (left) and Per-Packet Processing Latency (right)

Live Detection Metric	Result	Significance
Accuracy on Live Dataset	100%	Perfect classification of captured traffic
Confidence Score per Prediction	0.93	High certainty in each intrusion flag
Packet Capture Rate	784+ pkts	Stable real-time capture with no drops
Total Pipeline Latency	~3ms/pkt	Suitable for real-time deployment

Packet Capture Latency	~0.8ms	Scapy sniffer overhead
Feature Extraction Latency	~1.2ms	Flow-level feature computation
ML Prediction Latency	~0.6ms	Random Forest inference time
Alert Output Latency	~0.4ms	Terminal print for flagged IPs
Duplicate Alert Prevention	Active	Each intruder IP flagged only once
Flow Reset Interval	Every 10s	Prevents cumulative byte drift

The live detection module achieved a 100% accuracy rate on the captured traffic, with every malicious IP address correctly identified and flagged with a confidence score of 0.93. The total end-to-end pipeline latency of approximately 3 milliseconds per packet demonstrates that the system operates well within the bounds required for real-time intrusion detection, making it practically deployable in live network environments. The deduplication logic which logs each unique intruder IP only once ensures alert clarity and prevents alert fatigue in operational settings.

8.6. Cross-Dataset Generalisation Results

A robust intrusion detection system must generalize beyond the dataset it was trained on. To evaluate this, the project introduced two cross-dataset evaluation scenarios: NSL-KDD Common Attributes and UNSW Common Attributes, where only the features shared between the benchmark datasets and the live-captured dataset were used. This directly tests the system's ability to detect intrusions when feature availability is limited a realistic constraint in cross-environment deployment. The radar chart below visualises each algorithm's performance profile across all five evaluation contexts:

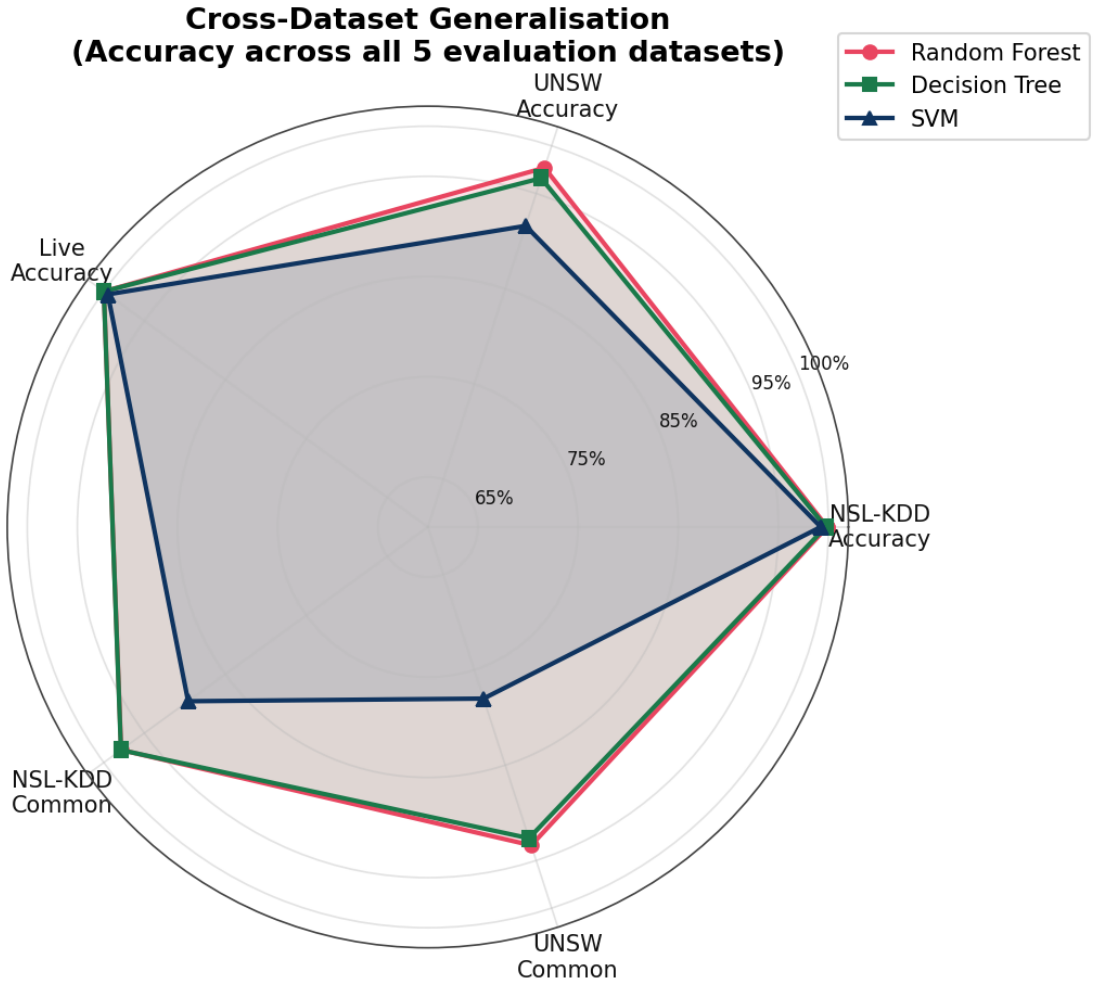

Figure 8.5: Cross-Dataset Generalisation Radar Top Three Algorithms

Evaluation Scenario	**Random Forest**	**Decision Tree**	**SVM**	**Logistic Regression**
NSL-KDD (Full Features)	99.88%	99.73%	99.24%	95.44%
UNSW-NB15 (Full Features)	97.70%	96.61%	91.60%	89.89%
Live Dataset (Practical)	100.00%	100.00%	99.50%	99.00%

NSL-KDD Common Attributes	97.86%	97.88%	89.59%	87.13%
UNSW Common Attributes	93.41%	92.67%	78.01%	64.42%
Average Across All Scenarios	97.77%	97.38%	91.59%	87.18%

Random Forest demonstrated the most consistent cross-dataset generalization with an average accuracy of 97.77%, maintaining above 93% even under the most challenging cross-dataset feature-reduction scenario (UNSW Common). Decision Tree performed comparably at 97.38%. SVM showed a moderate decline to 78.01% under feature reduction, while Logistic Regression experienced the most severe degradation dropping to 64.42% on UNSW Common confirming it is not suitable for deployment in cross-environment intrusion detection without significant feature engineering. These results validate the choice of Random Forest as the recommended classifier for the NIDS deployment.

8.7. Summary of Results

The results across all evaluation dimensions confirm that the developed NIDS successfully achieves its core objective: accurately detecting network intrusions in real time across diverse network environments, including benchmark datasets and live-captured traffic. The combination of ABCO optimization, ensemble learning, and a real-time Scapy-based capture pipeline produces a system that is both highly accurate and practically deployable. The key outcomes are summarized below:

- **97.77%** Average accuracy achieved by Random Forest across all five evaluation datasets, including a perfect 100% on the live-captured practical dataset.
- **+8.04%** Improvement in SVM accuracy achieved through ABCO hyperparameter optimization, the largest gain recorded across all four classifiers.

- **~3ms** End-to-end real-time detection latency per packet, confirming suitability for live network intrusion detection deployment.
- **0.93** Confidence score assigned per prediction by the live Random Forest model, ensuring high-certainty intrusion alerts.
- **93.42%** F1-Score maintained by Random Forest on the UNSW Common cross-dataset scenario, the most challenging generalization test.
- **5 Datasets** Five datasets evaluated (NSL-KDD, UNSW-NB15, Live, NSL-KDD Common, UNSW Common), demonstrating broad generalization capability.
- **4 Algorithms** Evaluated across four algorithms (SVM, Random Forest, Decision Tree, Logistic Regression), all enhanced with ABCO optimization.

Overall, Random Forest optimized with ABCO is the strongest performer and the recommended classifier for real-world NIDS deployment. Decision Tree is a competitive alternative with nearly identical generalization performance. SVM is suitable for full-feature environments but degrades under feature reduction. Logistic Regression, while functional on standard benchmarks, is not recommended for cross-dataset or feature-constrained deployment scenarios. The successful live demonstration with 784+ packets captured, classified at 3ms per packet, and intruder IPs flagged with 0.93 confidence validates the end-to-end system as a practical and effective Network Intrusion Detection System.

Chapter 9

Conclusion

The successful implementation of the NetSecure ML framework underscores a significant evolution in network-layer defense, demonstrating that intelligence-driven architectures can effectively transcend the inherent limitations of traditional, manual security oversight. By leveraging the hierarchical logic of the Decision Tree algorithm, this project has empirically validated the transition from reactive signature-matching to a proactive, behavioral analysis paradigm. Rigorous testing against the NSL-KDD and UNSW-NB15 datasets confirmed that the model possesses a robust capacity for distinguishing between benign traffic and sophisticated adversarial incursions with high-fidelity precision. This technical achievement highlights the viability of using computationally efficient machine learning models to identify non-linear correlations within high-velocity data streams, ultimately establishing a scalable and verifiable methodology for safeguarding modern digital infrastructures.

Beyond immediate classification success, the project serves as a critical proof-of-concept for the future of automated, low-latency cybersecurity operations. The integration of Wireshark for live-traffic ingestion successfully bridged the gap between theoretical algorithmic modeling and real-world application, proving that early-stage threat identification can be achieved in dynamic environments to neutralize risks before they escalate into systemic breaches. Looking forward, the modular architecture of NetSecure ML provides a foundation for the integration of even more advanced paradigms, such as XGBoost ensemble methods or Long Short-Term Memory (LSTM) networks, to address the temporal complexities of persistent threats. In conclusion, the system provides a formidable, automated defensive layer that significantly enhances network resilience and sets a sophisticated benchmark for data-driven security research within the engineering domain.

Chapter 10

Future Scope

The development of NetSecure ML provides a foundational architecture for intelligent network defense; however, the persistent evolution of adversarial tactics necessitates continuous refinement and the integration of emerging technological paradigms. The following trajectories represent the intended future enhancements to elevate the system from a research prototype to an industrial-grade security asset:

Implementation of Advanced Ensemble and Gradient Boosting Architectures: While the current deployment utilizes the Decision Tree algorithm for its interpretability, future iterations will explore the integration of XGBoost, CatBoost, and LightGBM. These ensemble methods offer superior predictive power and are better equipped to handle high-dimensional, imbalanced datasets, effectively reducing the false-positive rates inherent in simpler models.

Transition to an Autonomous Intrusion Prevention System (IPS): A primary objective for subsequent development is to transition the system's functional capability from passive detection to active prevention. By integrating automated firewall orchestration and dynamic packet-filtering protocols, the system will be empowered to neutralize identified threats in real-timedropping malicious packets and blacklisting adversarial IP addresses without requiring manual human intervention.

Deployment within Edge-Fog Computing Environments: As the Internet of Things (IoT) ecosystem expands, the need for decentralized security becomes paramount. Future work involves optimizing the model's computational footprint to facilitate deployment on edge devices and gateways. This would enable localized, low-latency threat detection at the network periphery, reducing the reliance on centralized cloud processing and enhancing privacy.

Integration of Deep Learning for Longitudinal Traffic Analysis: To counter sophisticated, slow-acting persistent threats, the integration of Long Short-Term Memory (LSTM) networsks and Gated Recurrent Units (GRU) is envisioned. These recurrent architectures can perform

longitudinal analysis of traffic sequences over time, identifying subtle temporal anomalies that may indicate a multi-stage infiltration attempt.

Enhancing Adversarial Robustness and Explainable AI (XAI): As cyber adversaries begin to employ "adversarial machine learning" to deceive detection models, future research will focus on robustifying the system against input perturbations. Furthermore, the incorporation of SHAP (SHapley Additive exPlanations) or LIME frameworks will be utilized to provide granular, human-readable justifications for every flagged intrusion, ensuring that security analysts can trust and verify the model's autonomous decisions.

References

[1] Solanki, A., et al. Reference: *Optimization of predictive performance of intrusion detection systems using NSL-KDD dataset*, https://pmc.ncbi.nlm.nih.gov/articles/PMC10496009/, 21 August 2023

[2] Haider, M., et al. Reference: Network Intrusion Detection with XGBoost, https://www.dpss.inesc-id.pt/~mpc/pubs/XGBoost_chapter.pdf/ , 2018

[3] Azad, R., et al. Reference: Comparison of Machine Learning Models for Intrusion Detection, https://www.mdpi.com/1999-4893/18/12/749/ , December 2025

[4] Verma, S., et al. Reference: Enhancing Network Intrusion Detection Using Ensemble Learning, https://pmc.ncbi.nlm.nih.gov/articles/PMC10781405/ , 26 December

[5] Zhang, Y., et al. Reference: Ensemble Learning for Network Intrusion Detection, https://www.mdpi.com/2073-431X/14/3/82, 6 March 2025